

Project GlobalChainX

Worldwide Supply Chain Resilience Platform

DevOps Engineering Documentation

Continuous Integration, Containerization, Orchestration, and Observability for a Cloud-Native
Logistics System

Prepared by: Swaraj Wattamwar

Course: DevOps — Semester 4

Github - https://github.com/Swaraj2259/devops_projects

Table of Contents

1. Introduction and Problem Statement
2. Objectives of the Project
3. Technology Stack Overview
4. System Architecture
5. Application Layer — Design and Development
6. Containerization with Docker
7. Infrastructure as Code with Terraform
8. Container Orchestration with Kubernetes
9. Continuous Integration and Continuous Deployment with Jenkins
10. Monitoring and Observability with Prometheus and Grafana
11. End-to-End Workflow Summary
12. Conclusion

1. Introduction and Problem Statement

GlobalChainX Logistics Corporation operates one of the largest interconnected supply chain ecosystems in the world, coordinating manufacturers, suppliers, transportation providers, warehouses, customs authorities, retailers, and financial institutions on a single platform. The system is expected to process a continuous stream of purchase orders, inventory transactions, shipping records, customs declarations, and logistics events around the clock.

This project addresses that problem by rebuilding the operational backbone of GlobalChainX using modern, cloud-native DevOps practices. Rather than relying on manually configured servers and ad-hoc deployment scripts, the platform is rebuilt around infrastructure automation, containerization, orchestration, automated delivery pipelines, and real-time observability — the same set of practices used by large-scale technology companies to keep critical systems available and recoverable.

2. Objectives of the Project

The project is built around the following objectives, each of which corresponds to a specific tool or practice in the DevOps lifecycle:

- Build a working application that represents the core function of the platform — a supply chain operations dashboard providing visibility into shipments, inventory, and operational alerts.
- Package the application using Docker so that it runs identically across any environment, removing the classic “it works on my machine” problem.
- Provision underlying cloud infrastructure using Terraform, following an Infrastructure as Code approach rather than manual console configuration.
- Deploy and manage the application using Kubernetes, enabling automated scaling, self-healing, and consistent rollout behaviour.
- Automate the build, test, and deployment process using a Jenkins Continuous Integration / Continuous Deployment (CI/CD) pipeline, removing manual deployment steps.
- Establish observability over the running system using Prometheus for metrics collection and Grafana for visualization, so that operational health can be monitored in real time.

Together, these objectives demonstrate the full DevOps lifecycle: code, build, ship, run, and monitor — applied to a realistic supply chain scenario.

3. Technology Stack Overview

Before describing the workflow, it is useful to understand what each tool in the stack is responsible for and why it was chosen. The table below summarizes the role of every major technology used in this project.

Tool	Category	Role in this project
Node.js / Express	Application runtime	Serves the supply chain dashboard and exposes a /metrics endpoint for monitoring.
Docker	Containerization	Packages the application and its dependencies into a single, portable image.
Terraform	Infrastructure as Code	Declaratively provisions cloud infrastructure in a repeatable, version-controlled way.
Kubernetes	Container orchestration	Runs, scales, and self-heals the application's containers automatically.
Jenkins	CI/CD automation	Automates the build → test → push → deploy pipeline whenever new code is pushed.
Prometheus	Metrics collection	Continuously scrapes application and cluster metrics for monitoring.
Grafana	Visualization	Turns Prometheus metrics into readable dashboards for operational visibility.
GitHub	Version control	Acts as the single source of truth for all source code and pipeline configuration.
DockerHub	Image registry	Stores built Docker images so they can be pulled by the deployment environment.

4. Application Layer — Design and Development

4.1 Purpose of the Application

The application represents the operational dashboard that supply chain staff would use to monitor the health of the network in real time. Rather than building every microservice described in the case study (customs processing, warehouse management, financial settlement, and so on), the project focuses on building one representative, fully functional service end-to-end and proving that it can be carried through the entire DevOps lifecycle — from source code to a monitored, running deployment. This mirrors how real organizations validate a DevOps platform: by piloting it on one service before rolling it out everywhere.

4.2 Technology Choice

The dashboard is built using Node.js with the Express framework. Node.js was chosen because it is lightweight, starts quickly inside a container, and has a minimal set of dependencies, which

keeps the resulting Docker image small and the deployment fast. Express is used to serve both the dashboard's HTML interface and a dedicated /metrics endpoint that exposes operational counters in Prometheus's plain-text exposition format.

4.3 Dashboard Features

The dashboard surfaces the information that matters most to a supply chain operations team:

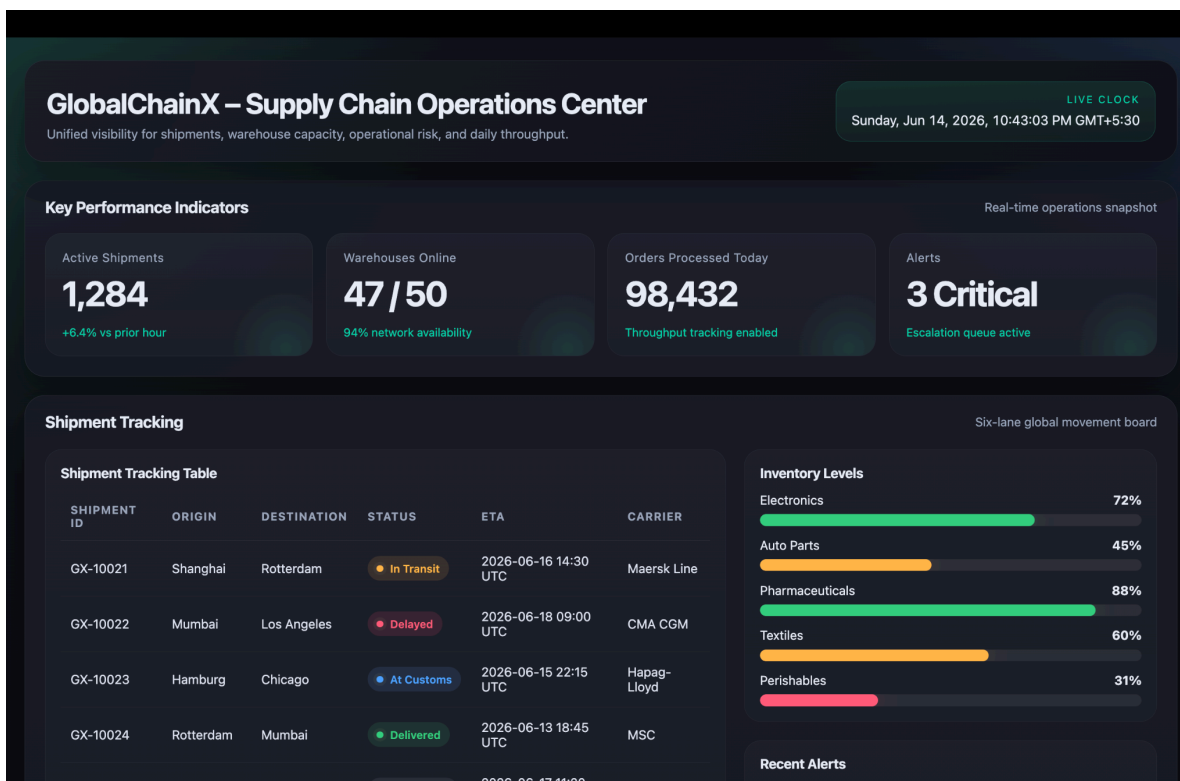


Figure 5.1 — GlobalChainX Supply Chain Operations Center dashboard, running live and accessed through the deployed service.

This screenshot is the proof point for the entire project: it shows the application not running from a developer's local machine, but actually being served through the full chain of Docker, Kubernetes, and the Service abstraction described in later sections.

5. Containerization with Docker

5.1 Why Containerization Matters

Before containerization became standard practice, deploying an application reliably across different environments (a developer's laptop, a test server, and a production server) was a recurring source of failure. Differences in operating system version, installed libraries, and

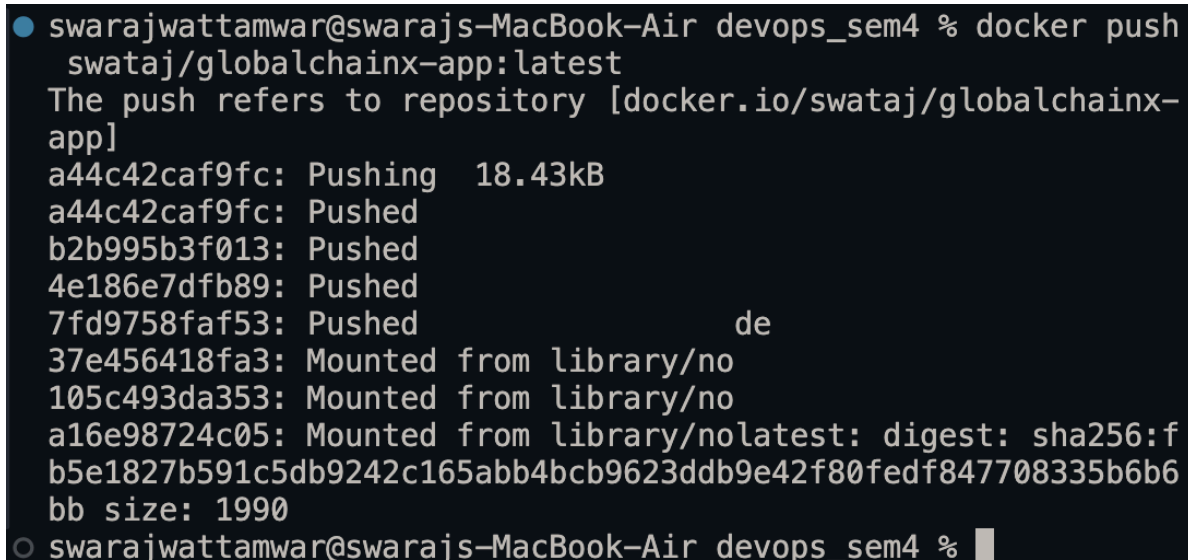
configuration would cause an application to behave correctly in one environment and fail in another.

6.2 Dockerfile Design

The project's Dockerfile uses a slim Node.js base image to minimize size and attack surface. The build process follows a deliberate order: dependency manifests (package.json) are copied and installed before the rest of the application source code is copied in. This ordering takes advantage of Docker's layer caching — if the source code changes but dependencies do not, Docker can reuse the cached dependency-installation layer instead of reinstalling everything from scratch, which significantly speeds up repeated builds during development and in the CI/CD pipeline.

6.3 Building and Publishing the Image

Once built locally, the image is tagged and pushed to DockerHub, which acts as a central registry. Publishing the image to a registry is what allows Kubernetes — running in a separate environment from the developer's machine — to pull and run that exact image without needing access to the original source code or build environment.



```
● swarajwattamwar@swarajs-MacBook-Air devops_sem4 % docker push
  swataj/globalchainx-app:latest
The push refers to repository [docker.io/swataj/globalchainx-
app]
a44c42caf9fc: Pushing  18.43kB
a44c42caf9fc: Pushed
b2b995b3f013: Pushed
4e186e7dfb89: Pushed
7fd9758faf53: Pushed                                de
37e456418fa3: Mounted from library/no
105c493da353: Mounted from library/no
a16e98724c05: Mounted from library/nolatest: digest: sha256:f
b5e1827b591c5db9242c165abb4bcb9623ddb9e42f80fedf847708335b6b6
bb size: 1990
○ swarajwattamwar@swarajs-MacBook-Air devops_sem4 %
```

Figure 5.1 — Docker image being pushed to the DockerHub registry, making it available for deployment.

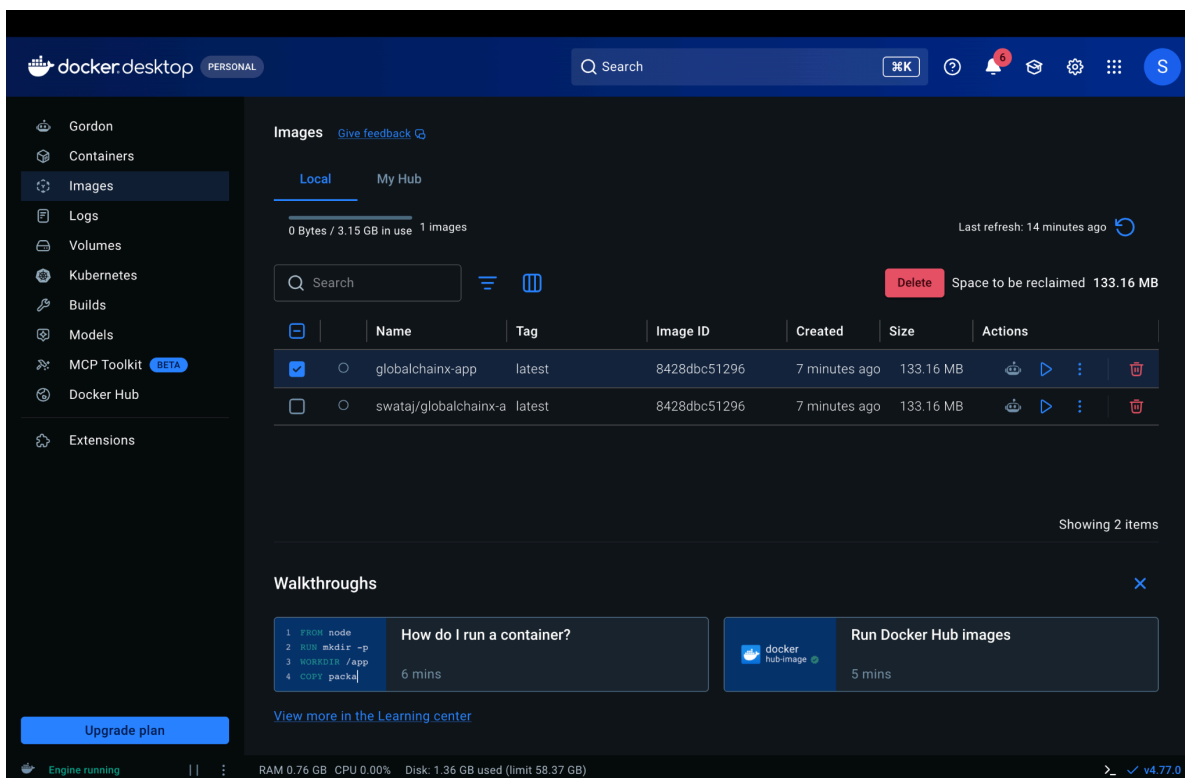


Figure 6.2 — Docker Desktop showing the locally built GlobalChainX application image and its size.

Containerization is the foundation that every later stage in this project depends on. Terraform provisions the infrastructure that infrastructure could run on; Kubernetes orchestrates containers, not raw application code; and Jenkins automates the build-and-push cycle for exactly this kind of image. Without a properly built and published image, none of the later automation has anything to deploy.

6. Infrastructure as Code with Terraform

6.1 The Problem with Manual Infrastructure

Traditionally, provisioning cloud infrastructure meant manually clicking through a cloud provider's web console — creating a virtual machine, configuring a security group, attaching a key pair, and noting down the resulting IP address. This approach does not scale: it is slow, undocumented, easy to get wrong, and impossible to reliably repeat. If the environment needs to be destroyed and rebuilt, or replicated in another region, the entire manual process has to be repeated from memory.

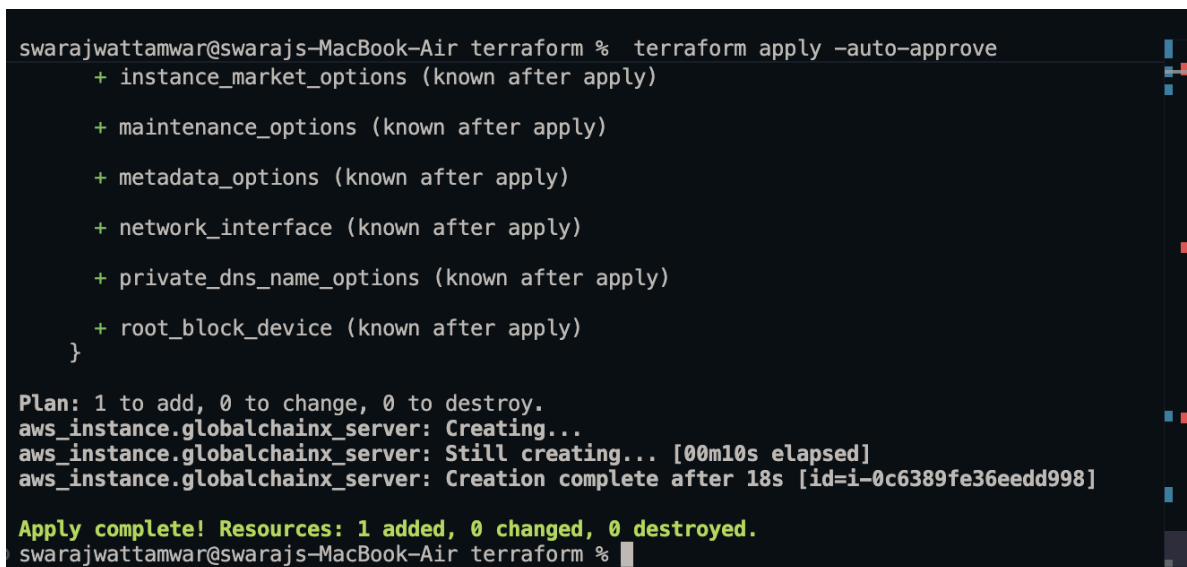
6.2 Infrastructure as Code

Terraform solves this by letting infrastructure be described declaratively, in code, rather than configured by hand. A Terraform configuration file specifies what resources should exist — for example, a virtual machine of a certain type, in a certain region, with certain network rules — and Terraform's engine figures out how to create, update, or remove real cloud resources to match that description. This has several benefits that matter directly to the resilience goals of GlobalChainX:

- Repeatability — the exact same infrastructure can be recreated identically at any time, which is essential for disaster recovery.
- Version control — infrastructure definitions live in source control alongside application code, so changes are tracked and reviewable.
- Consistency — there is no risk of manually configuring two environments slightly differently.

6.3 What Was Provisioned

In this project, Terraform is used to declaratively provision the underlying cloud compute resource and its associated networking rules — defining the access ports required by the rest of the stack (the application port, the CI/CD server port, and the monitoring ports) as part of the infrastructure definition itself, rather than configuring them by hand afterward. Running `terraform apply` takes that declarative definition and creates the corresponding real infrastructure in a single, repeatable command.

A terminal window on a MacBook Air showing the execution of the Terraform apply command. The prompt is 'swarajwattamwar@swarajs-MacBook-Air terraform %'. The command entered is 'terraform apply -auto-approve'. The output shows a list of resources to be added, including instance_market_options, maintenance_options, metadata_options, network_interface, private_dns_name_options, and root_block_device. It then shows the plan: 1 to add, 0 to change, 0 to destroy. The resource 'aws_instance.globalchainx_server' is shown as 'Creating...', then 'Still creating...' with a 10-second elapsed time, and finally 'Creation complete after 18s' with an ID. The final output is 'Apply complete! Resources: 1 added, 0 changed, 0 destroyed.' and the prompt returns to 'swarajwattamwar@swarajs-MacBook-Air terraform %'.

```
swarajwattamwar@swarajs-MacBook-Air terraform % terraform apply -auto-approve
+ instance_market_options (known after apply)
+ maintenance_options (known after apply)
+ metadata_options (known after apply)
+ network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_instance.globalchainx_server: Creating...
aws_instance.globalchainx_server: Still creating... [00m10s elapsed]
aws_instance.globalchainx_server: Creation complete after 18s [id=i-0c6389fe36eedd998]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
swarajwattamwar@swarajs-MacBook-Air terraform %
```

Figure 7.1 — Terraform applying the infrastructure configuration and completing successfully.

This single command-driven provisioning process is the core demonstration of Infrastructure as Code: the same configuration file used here could be reapplied to recreate the identical

environment after a disaster, or adapted slightly to provision a second, parallel environment for testing — both without manual console work.

8. Container Orchestration with Kubernetes

8.1 Why Orchestration Is Necessary

Running a single Docker container manually is simple. Running many containers reliably, at scale, with automatic recovery from failure, is not. Kubernetes is a container orchestration platform that takes a declarative description of how an application should run — how many copies, what resources it needs, how to check if it is healthy — and continuously works to keep the real state of the cluster matching that description.

This directly addresses one of the core weaknesses identified in the original case study: services experiencing outages with no automated recovery. Kubernetes is designed specifically to detect failed containers and replace them automatically, without human intervention.

8.2 Key Concepts Used in This Project

- Pod — the smallest deployable unit in Kubernetes, representing one running instance of the application container.
- Deployment — a higher-level object that manages a desired number of identical pod replicas, and handles rolling updates when a new image version is deployed.
- Service — a stable network identity and load-balancing layer in front of a set of pods, so that other parts of the system (or end users) do not need to know the changing IP addresses of individual pods.
- Liveness and readiness probes — health checks that Kubernetes uses to determine whether a pod is functioning correctly and ready to receive traffic; a failing probe triggers an automatic restart.

8.3 Deployment Configuration

The application is deployed with two replica pods rather than one. Running multiple replicas means that if one pod fails or is being restarted, the other continues to serve traffic, and the Service automatically routes around the unhealthy instance. Resource requests and limits are also defined, so the cluster scheduler can make informed decisions about where to place the pods and prevent any single pod from consuming excessive resources.

8.4 Verifying the Deployment

After applying the Deployment and Service manifests, the cluster's state can be inspected directly using the Kubernetes command-line tool, `kubectl`. The pods transition from a creating state to a running state once their containers start successfully and pass their health checks.

```
swarajwattamwar@swarajs-MacBook-Air devops_sem4 % kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
globalchainx-app-69d4b85f49-qxp7p  0/1     ContainerCreating   0           14s
globalchainx-app-69d4b85f49-tb6hp  0/1     ContainerCreating   0           14s
swarajwattamwar@swarajs-MacBook-Air devops_sem4 % kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
globalchainx-service NodePort    10.96.128.145 <none>         3000:30000/TCP   28s
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP          48s
swarajwattamwar@swarajs-MacBook-Air devops_sem4 %
```

Figure 8.1 — `kubectl` confirming both application pods and the NodePort Service are active inside the cluster.

This Service is what later allows Prometheus to discover and scrape the application's metrics endpoint, and what allows the application to be reached in a browser — both described in the following sections.

9. Continuous Integration and Continuous Deployment with Jenkins

9.1 The Case for Automation

Without automation, deploying a new version of an application requires a person to manually build the Docker image, manually push it to a registry, and manually update the Kubernetes deployment — a process that is slow, error-prone, and does not scale as a team or system grows. Continuous Integration and Continuous Deployment (CI/CD) automates this entire sequence so that pushing code to version control is the only manual step required; everything after that happens automatically and consistently.

9.2 Pipeline Design

The project uses a Jenkins Pipeline, defined declaratively in a Jenkinsfile stored in the same Git repository as the application source code. Defining the pipeline as code, rather than configuring it through Jenkins's user interface, means the deployment process itself is version-controlled, reviewable, and reproducible — the same Infrastructure as Code principle applied to the delivery process rather than to the infrastructure.

The pipeline is structured into five sequential stages, each with a single clear responsibility:

Stage	Responsibility
Checkout	Pulls the latest application source code from the GitHub repository.

Build Docker Image	Builds a new Docker image from the checked-out source using the Dockerfile.
Push to DockerHub	Authenticates with DockerHub and pushes the newly built image as a versioned artifact.
Deploy to Kubernetes	Applies the Kubernetes manifests and updates the running Deployment to use the new image.
Verify Deployment	Waits for the rollout to complete successfully and reports the resulting pod and service state.

Credentials such as the DockerHub login and the Kubernetes cluster configuration are never written directly into the pipeline code. Instead, they are stored securely inside Jenkins's credential store and only injected into the pipeline at the moment they are needed, which keeps secrets out of source control entirely.

9.3 Pipeline Execution

When triggered, Jenkins works through all five stages in sequence. A successful run, with every stage passing, is the clearest possible evidence that the entire build-to-deployment chain is functioning correctly end to end — not just that the code compiles, but that it can actually be built into an image, published, and rolled out to a live cluster automatically.

The screenshot displays the Jenkins web interface for a pipeline named 'GlobalChainX-Pipeline'. The current stage is '#14', which has completed successfully. The pipeline consists of seven stages: Checkout SCM, Checkout, Build Docker Image, Push to DockerHub, Deploy to Kubernetes, Verify Deployment, and Post Actions. All stages are marked with green checkmarks, indicating success. The 'Post Actions' stage is currently selected, showing a list of steps: 'docker logout || true' (0.64s) and 'Pipeline completed successfully — GlobalChainX deployed.' (40ms). The overall pipeline status is 'Completed successfully'.

Jenkins / GlobalChainX-Pipeline / #14 / Stages

#14 Started by Swaraj Wattamwar Started 6 min 2 sec ago Queued 13 ms Took 52 sec Rerun Settings

Start — End

Checkout SCM — Checkout — Build Docker Image — Push to DockerHub — Deploy to Kubernetes — Verify Deployment — Post Actions

Post Actions 0.72s Started 5m 20s ago Jenkins

- ✓ Checkout SCM 0.92s
- ✓ Checkout 0.88s
- ✓ Build Docker Image 4s
- ✓ Push to DockerHub 11s
- ✓ Deploy to Kubernetes 1s
- ✓ Verify Deployment 29s
- ✓ Post Actions 0.72s

✓ docker logout || true 0.64s

✓ Pipeline completed successfully — GlobalChainX deployed. 40ms

0 ✓ Pipeline completed successfully — GlobalChainX deployed.

Figure 9.1 — Jenkins pipeline completing all five stages successfully, with detailed stage timing and post-build actions.

This automated pipeline is what directly answers the case study's requirement for rapid recovery: if a disruption requires rolling back to a previous version or redeploying after a fix, the same pipeline can be re-run rather than requiring a manual, multi-step recovery procedure.

10. Monitoring and Observability with Prometheus and Grafana

10.1 Why Observability Is a Core Requirement

One of the central weaknesses highlighted in the original case study was limited operational visibility — outages were discovered reactively rather than detected proactively. Observability tooling exists to close that gap: it continuously collects data about how a system is behaving, so that operations teams can detect anomalies, investigate incidents, and verify that recovery actions actually worked.

10.2 Prometheus — Metrics Collection

Prometheus is a metrics collection and storage system that works on a pull-based model: rather than waiting for applications to send it data, Prometheus actively reaches out to known targets at a regular interval and scrapes their current metric values from a `/metrics` HTTP endpoint. The application built for this project exposes exactly such an endpoint, returning live counters for active shipments, warehouses online, daily orders processed, and critical alert counts, in the plain-text format Prometheus expects.

To tell Prometheus where to find this endpoint inside the Kubernetes cluster, the project defines a `ServiceMonitor` resource — a Kubernetes-native configuration object, understood by the Prometheus Operator, that declares which Service to watch and which named port and path to scrape. This is a deliberately declarative approach: rather than manually editing Prometheus's configuration file, the scrape target is described as a Kubernetes object that lives alongside the application's own Deployment and Service manifests.

Once configured, Prometheus's own target health page can be used to confirm that scraping is actually happening, and that every target is reporting a healthy, up state.

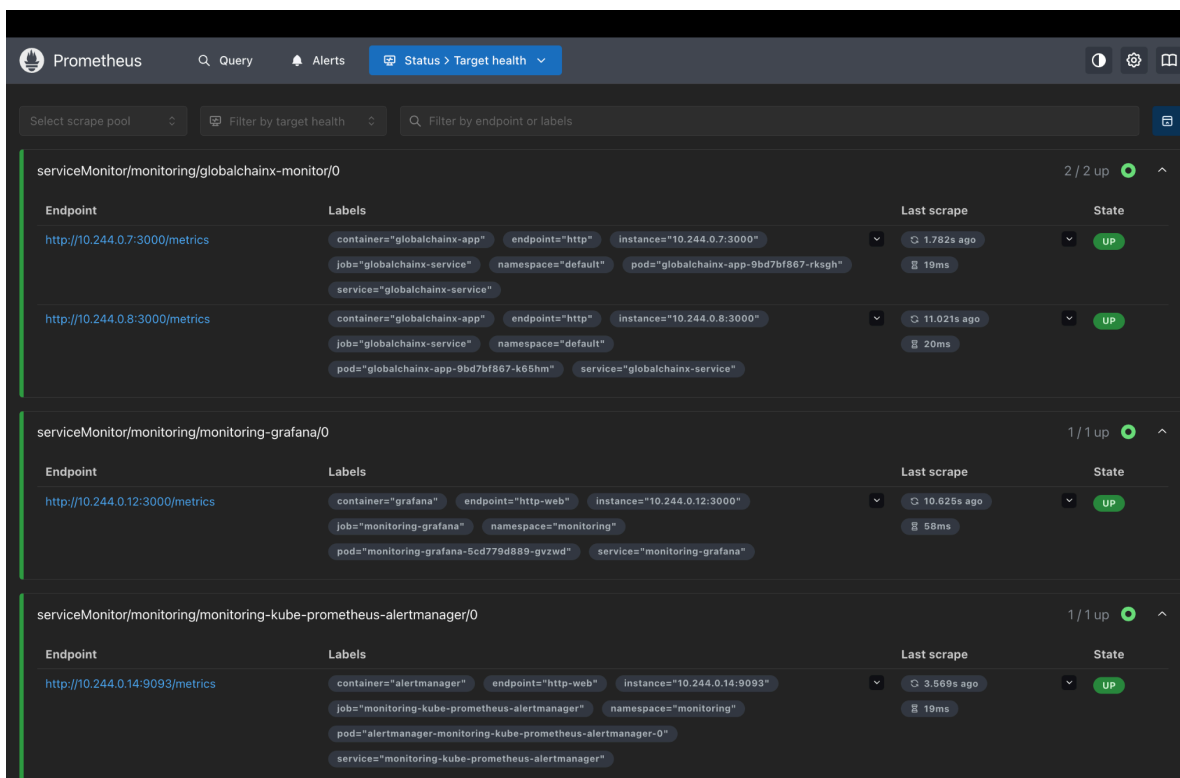


Figure 10.1 — Prometheus target health page confirming both application pods are being scraped successfully and reporting an up state.

10.3 Grafana — Visualization

Raw metric data on its own is not very useful to a human operator. Grafana solves this by connecting to Prometheus as a data source and rendering its metrics as readable, continuously refreshing dashboards. A dashboard is composed of individual panels, each backed by a Prometheus query, and arranged together to give a single-glance view of system health.

For this project, a dedicated operations dashboard was built containing four panels, each corresponding to one of the application's core KPIs: active shipments, warehouses online, orders processed today, and critical alerts. Because the underlying metric values are scraped directly from the live application pods, the dashboard reflects the actual current state of the running system rather than a static snapshot.

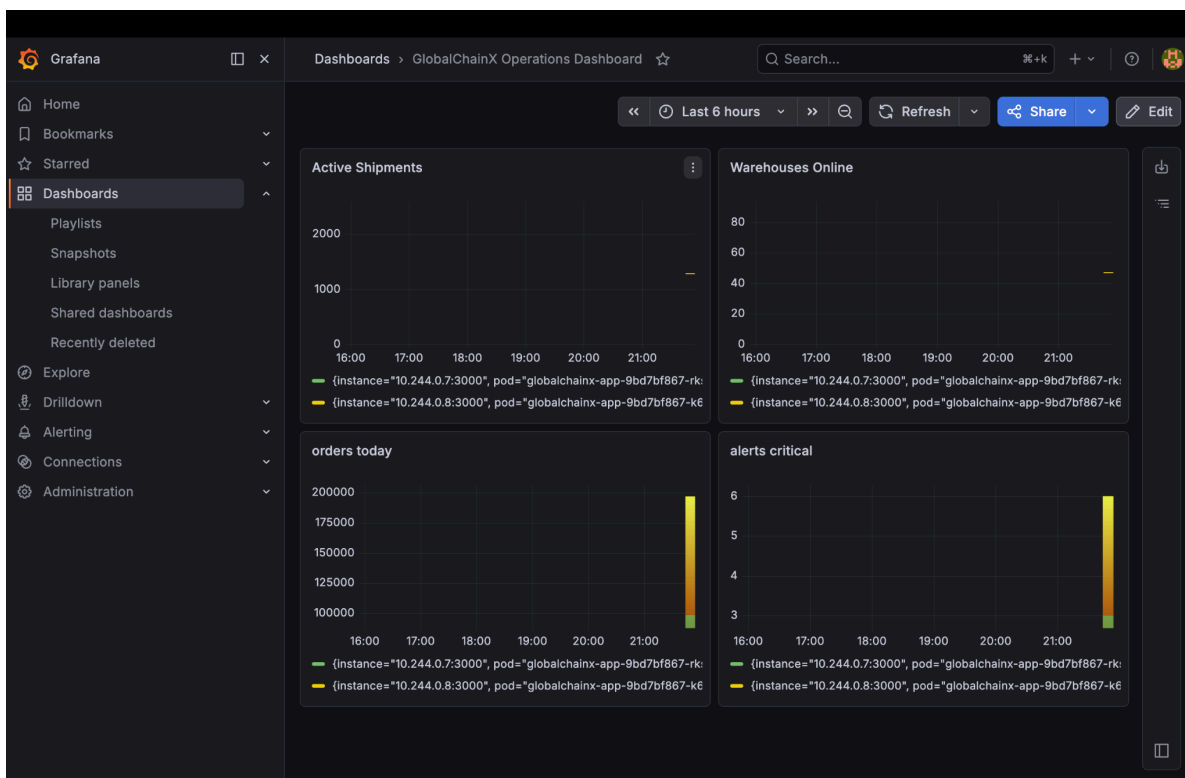


Figure 10.2 — GlobalChainX Operations Dashboard in Grafana, visualizing live metrics scraped from both application pods.

Together, Prometheus and Grafana close the observability loop described in the project objectives: Prometheus answers “what is the current state of the system,” and Grafana answers “how do I see that state at a glance.” This combination is what allows an operations team to notice a problem — for example, a drop in warehouses online or a spike in critical alerts — long before it would otherwise be reported by an end user.

11. End-to-End Workflow Summary

Bringing every preceding section together, the complete operational workflow of the GlobalChainX platform, from a code change to a monitored deployment, is as follows:

1. Application code for the supply chain dashboard is written and committed to a GitHub repository, the single source of truth for the project.
2. Jenkins, watching that repository, checks out the latest code at the start of every pipeline run.
3. Jenkins builds a Docker image from the application's Dockerfile, packaging the Node.js application and its dependencies into a single, portable artifact.
4. Jenkins authenticates with DockerHub and pushes the newly built image, making it available as a pullable artifact tagged with the current build number.

5. Jenkins applies the project's Kubernetes manifests and updates the running Deployment to reference the newly pushed image, triggering a rolling update.
6. Kubernetes pulls the new image from DockerHub and starts new pod replicas, replacing the old ones while keeping the Service continuously available.
7. Jenkins's final pipeline stage verifies that the rollout completed successfully and that the expected number of pods are running and healthy.
8. Prometheus, configured via a ServiceMonitor, continuously scrapes the live application pods' /metrics endpoint, collecting up-to-date operational counters.
9. Grafana queries Prometheus and renders those counters on the GlobalChainX Operations Dashboard, giving the operations team continuous, real-time visibility into the platform's health.

Separately, Terraform is used to declaratively provision the underlying cloud compute resource that the platform's infrastructure depends on, following the same Infrastructure as Code principle applied throughout the rest of the stack.

Each stage in this workflow is independently automated and independently verifiable — a property that maps directly back to the project's founding objective: replacing a fragmented, manually-operated infrastructure with one that is automated, observable, and quick to recover.

12. Conclusion

This project demonstrates a complete, working DevOps pipeline for a representative service within the GlobalChainX supply chain platform. Starting from a single Node.js application, the project applies the core practices of modern DevOps engineering in sequence: containerization with Docker to guarantee consistent execution across environments; Infrastructure as Code with Terraform to make infrastructure provisioning repeatable and version-controlled; container orchestration with Kubernetes to enable automated scaling and self-healing; continuous integration and deployment with Jenkins to remove manual steps from the release process; and observability with Prometheus and Grafana to give the operations team real-time insight into system health.

Each of these tools was chosen because it solves a specific weakness identified in the original case study — fragmented infrastructure, inconsistent deployments, limited visibility, and slow recovery from disruption. The result is a pipeline in which a single code change can move automatically and reliably from a developer's commit to a running, monitored deployment, with no manual deployment steps in between.

While this implementation focuses on proving the pipeline end to end on one core service rather than rebuilding every microservice described in the original case study, the same architecture, tooling, and automation pattern demonstrated here is directly extensible to the platform's other

services — customs processing, warehouse management, and supplier communication — without requiring a different approach.